Automated Detection of Rewritten Text in Student Reports

A submitted report for the module on Information Integrity

1. Introduction

In recent years, the volume of scholarly publishing has increased rapidly, and institutions have responded by deploying automated integrity screening tools. Most of these systems rely on comparing character sequences. Unfortunately, a learner who substitutes half of the words within a paragraph can push a string-matching score close to zero while leaving the original claim untouched.

Semantic representations provide a different route: a transformer encoder maps a sentence to a dense vector in a continuous space, and sentences that share meaning occupy neighbouring regions of that space. This property is precisely what paraphrase detection requires.

2. Background

Fingerprinting methods that hash n-grams are still widely used because they are quick and comparatively transparent. However, any edit invalidates the hashes, which limits their usefulness for rewritten text. Earlier studies of paraphrase identification reported that contextual embeddings tend to beat static word vectors on semantic textual similarity benchmarks, though the size of the improvement depends strongly on the domain under study. Prior work also reports a consistent problem: generic academic phrasing produces false positives in every embedding-based system evaluated so far.

3. Experimental Setup

Our pipeline encodes each sentence with a pretrained MiniLM encoder after segmentation with an abbreviation-aware splitter, and then scores every pair of sentences with cosine similarity. Pairs are classified as exact, near-exact or semantic based on lexical evidence that is computed independently of the encoder. Reference lists are excluded from matching because bibliographies repeat across papers and inflate similarity without indicating reuse.

4. Findings and Observations

The pipeline recovered 71% of human-labelled paraphrases at a similarity threshold of 0.62, while the character n-gram baseline recovered 18%. Exact matches were detected with equal reliability by both approaches. Precision fell when the threshold was lowered to 0.55, largely because methodology boilerplate began to match across unrelated papers. Taken together, the combination of lexical and semantic signals produced the best F1 score on our annotated set.

5. Broader Considerations

It is important to note that a cosine similarity of 0.8 does not mean that a passage was copied; it means the two sentences occupy a similar region of the embedding space. Shared definitions, standard methodological descriptions and common terminology all produce high scores legitimately. Any deployment of semantic similarity for integrity screening therefore requires human review of the flagged passages. Recent studies have shown that reviewers make better decisions when the interface displays the matched text alongside the score, so that context can be judged directly.

6. Word Embeddings and Vector Space Models

A word embedding represents each token as a vector of real numbers learned from co-occurrence statistics. Word2vec trains a shallow network to predict context words, while GloVe factorises a global co-occurrence matrix. Neither technique considers word order, so the resulting representation of a sentence is often an unweighted average of its token vectors. This aggregation discards syntax and cannot distinguish "the dog bit the man" from "the man bit the dog". Contextual encoders address this limitation by producing a different vector for each token occurrence.

7. Automated Assessment of Programming Assignments

Programming assignment grading tools typically execute student code against a suite of hidden test cases and report the proportion of tests that pass. Static analysis can additionally detect forbidden constructs, such as dynamic memory allocation or unbounded recursion. Neither approach observes the structure of the submitted solution, which means that two syntactically different programs implementing the same algorithm receive identical credit. Structure-aware grading requires comparing abstract syntax trees or program dependence graphs, a task that is considerably more expensive than running unit tests.

8. Conclusion

Semantic similarity is a useful complement to lexical matching rather than a replacement for it. Systems that report exact, near-exact and semantic evidence separately give reviewers the information they need to distinguish copying from shared vocabulary.

References

Smith JA, Kumar R. Fingerprinting methods for large document collections. Journal of Information Integrity. 2019;14(2):112-129.

Wallace T. Vector space models of meaning. Computational Semantics Review. 2018;3(1):15-40.